AI COPILOT CODE REVIEW USING LARGE LANGUAGE MODELS

A Mini Project Report Submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Or

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

Or

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

By

Bandi Srihitha – 2420030099 (TEAM LEAD)

Prathipati Pranathi – 2420030043

Emani Mahathi – 2420030093

Under the Esteemed Guidance of

Dr . KATANGURI SWANTHANA (CSE)



Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Or

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

Or

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

DECLARATION

The mini-project report entitled "**AI COPILOT CODE REVIEW USING LARGE LANGUAGE MODELS** " is a record of bona fide work carried out by Bandi Srihitha (2420030099), Prathipati Pranathi (2420030043), and Emani Mahathi (2420030093).in partial fulfillment of the requirements for the award of the B.Tech degree at K L (Deemed to be) University. The work presented in this report is original to the best of our knowledge, and all datasets, tools, code, and external sources used have been appropriately acknowledged.

Bandi Srihitha – 2420030099

Prathipati Pranathi – 2420030043

Emani Mahathi – 2420030093

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Or

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

Or

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE



CERTIFICATE

This is to certify that the mini-project report entitled "**AI COPILOT CODE REVIEW USING LARGE LANGUAGE MODELS**" is a bona fide work carried out by **Bandi Srihitha (2420030099), Prathipati Pranathi (2420030043), and Emani Mahathi (2420030093).**under the supervision of **Dr . KATANGURI SWANTHANA (CSE),** during the academic year [2026–2027], in partial fulfillment of the requirements for the award of the Bachelor of Technology degree at K L (Deemed to be) University.

Signature of the Project Supervisor

Signature of the HOD **Signature of the External Examiner**

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my guide, **Dr . KATANGURI SWANTHANA (CSE),**

Department of Computer Science which cannot be expressed in words for her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. P.Venkateshwar Sir (CSE)/Dr. Kaja Shareef Sir (CSIT)/Dr.Sreenath**, Head of the *Department for Computer Science Engineering/ Department for Computer Science and Information Technology/ Department Of Artificial Intelligence And Data Science* for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

Bandi Srihitha – 2420030099

Prathipati Pranathi – 2420030043

Emani Mahathi – 2420030093

TABLE OF CONTENTS

S.No.	Contents	Page no.
1.	Abstract	6
2.	Introduction	7
3.	Literature survey	8
4.	Problem Statement and Objectives	12
5.	Hardware and Software requirements	15
6.	Implementation	16
7.	Results & Discussion	23
8.	Conclusion & Future Scope	33
9.	References	36

1. ABSTRACT

Software code review is an essential software engineering activity used to identify defects, security concerns, coding-standard violations, and maintainability problems. Manual code review can be time-consuming and dependent on reviewer expertise. Traditional static-analysis tools identify predefined issues but have limited semantic and repository-level awareness. Large Language Models (LLMs) can generate explanations and code-review suggestions, but may produce generic or unsupported findings when relevant context is unavailable.

This project presents an AI Copilot for context-aware automated code review using language-specific static analysis, CodeBERT semantic embeddings, FAISS similarity search, Retrieval-Augmented Generation (RAG), and CodeLlama. The system supports Python, Java, and JavaScript through Pylint, PMD, and ESLint, respectively. CodeBERT generates semantic representations that are indexed using FAISS to retrieve relevant code context from CodeSearchNet and selected public GitHub repositories. The retrieved context and static-analysis findings are combined with the submitted source code to construct a prompt for CodeLlama. The generated review contains issue descriptions, severity, supporting evidence, confidence information, and suggested fixes.

The prototype evaluation reports overall static-analysis precision of 0.56, recall of 1.00, F1-score of 0.71, and accuracy of 0.56. Retrieval evaluation reports Recall@1 of 0.67, Recall@2 of 0.67, Recall@3 of 1.00, and mean reciprocal rank of 0.78. These results demonstrate the feasibility of combining static analysis, semantic retrieval, and LLM-based review in a unified application. Larger benchmark evaluations and human assessment are required to establish broader effectiveness.

Team Name:

AI COPILOT CODE REVIEW USING LARGE LANGUAGE MODELS

Team Members and Responsibilities:

1. Bandi Srihitha: Team Lead, Frontend and Integration,RAG Pipeline.
2. Prathipati Pranathi: AI/ML and RAG Development
3. Emani Mahathi: Dataset, Testing and Evaluation
4. CodeXGLUE -https://github.com/microsoft/CodeXGLUE.git
5. CodeSearchNet -https://github.com/github/CodeSearchNet.git
6. GitHub Public Repositories-https://github.com?utm_source=
7. Github Project Repository- https://github.com/SrihithaBandi99/KLH-CSE-2026-27-2420030099-AI-CodeReview.git

2. INTRODUCTION

2.1 Background

Software development involves frequent modifications to source code. Before code is integrated into a larger software project, it must be examined to identify programming errors, potential defects, security weaknesses, and violations of coding standards. This process is known as code review.

Manual code review helps improve software quality, but reviewing large amounts of code requires time and technical expertise. Traditional static-analysis tools automate part of this process by applying predefined rules. However, they may not understand the broader purpose of a program or the relationships between different files in a repository.

Large Language Models have introduced new possibilities for automated code review. They can understand code, explain potential issues, and recommend modifications. However, LLM-generated reviews may contain incorrect, generic, or unsupported findings when the model lacks sufficient context.

The proposed project combines static-analysis tools, semantic retrieval, and an LLM to provide context-aware code-review assistance.

2.2 Problem Domain

The project belongs to automated software engineering, natural language processing, code intelligence, and machine-learning-assisted software quality analysis.

The system processes source code and produces a structured natural-language review. The input consists of source code written in Python, Java, or JavaScript. The output includes identified issues, explanations, severity levels, evidence, and suggested fixes.

2.3 Proposed Approach

The proposed system follows these stages:

1. Source-code upload.
2. Programming-language detection.
3. Static analysis using the appropriate tool.
4. Code embedding generation using CodeBERT.
5. Similarity search using FAISS.
6. Context assembly using Retrieval-Augmented Generation.
7. Review generation using CodeLlama.
8. Structured reporting through a Streamlit interface.

The combination of deterministic analysis and LLM reasoning is intended to improve the relevance, explainability, and usability of generated reviews.

2.4 Target Users

The intended users include software developers, students learning programming, project teams, and reviewers who need assistance identifying potential code-quality issues.

2.5 Organization of the Report

This report presents the literature survey, problem statement, objectives, system requirements, implementation methodology, experimental results, discussion, conclusion, future scope, and references.

3. LITERATURE SURVEY

No.	Research Paper	Model / Method	Outcome	Limitations
1	AI-powered Code Review with LLMs: Early Results (2024)	GPT-4-based LLM framework	Identifies bugs, code smells, risks, and improvements.	Early-stage results; limited benchmark evidence.
2	An Insight into Security Code Review with LLMs (2024)	Six LLMs evaluated with CWE context	Shows potential for detecting security defects.	Verbose responses and sensitivity to code length and context.
3	Evaluating Source Code Quality with Large Language Models (2024)	GPT-3.5 Turbo, GPT-4o, and SonarQube	LLMs can assess certain code-quality aspects.	Limited to Java and two open-source projects.
4	Combining Large Language Models with Static Analyzers for Code Review Generation (2025)	Hybrid LLM, static analysis, and RAG	Improved relevance and completeness of code reviews.	False positives and irrelevant suggestions may occur.
5	CodeAgent: Autonomous Communicative Agents for Code Review (2024)	Multi-agent LLM system with QA checker	Supports code inconsistency, vulnerability, style, and revision tasks.	Increased complexity and computational overhead.
6	Fine-tuning and Prompt Engineering for LLMs-Based Code Review Automation (2024)	Fine-tuning and prompt engineering	Shows how task-specific tuning and prompting can improve automated reviews.	Requires suitable training data and careful configuration.
7	Automating Code Review Activities by Large-Scale Pre-training (2022)	CodeReviewer pretrained model	Improves review comment generation, quality estimation, and code refinement.	Requires large-scale review data and specialized pre-training.
8	Code Review Automation using Retrieval Augmented Generation (2025)	RARe: Retrieval-Augmented Reviewer	Retrieval improves review relevance compared with baselines.	Depends on retrieval quality and available knowledge.
9	LAURA: Enhancing Code Review Generation with Context-Enriched Retrieval-Augmented LLM (2025)	Context-enriched RAG with LLMs	Context and retrieved review examples improve review usefulness.	Requires high-quality context and historical review data.
10	Automated Code Review Using Large Language Models with Symbolic Reasoning (2025)	LLMs with symbolic reasoning and prompting	Combines reasoning with LLMs for automated code review.	More complex pipeline; results depend on the evaluation task.

3.1 CodeBERT: A Pre-Trained Model for Programming and Natural Languages

Feng et al. introduced CodeBERT, a pretrained model designed to learn representations of programming languages and natural language. It supports code-related tasks such as code search and understanding. Its learned representations can be used to compare code fragments based on semantic similarity. In this project, CodeBERT is used to create code embeddings for the retrieval stage. A limitation is that semantic embeddings alone do not identify every programming defect or guarantee that retrieved code is relevant.

3.2 CodeSearchNet Challenge

Husain et al. introduced the CodeSearchNet Challenge to evaluate semantic code-search systems. The resource contains code and associated natural-language information for code-search research. It provides useful material for retrieving code examples related to a query. The proposed system uses CodeSearchNet as a source of code context for semantic retrieval. However, code-search relevance does not necessarily mean that a retrieved example contains a verified defect or a correct code-review recommendation.

3.3 CodeXGLUE

Lu et al. presented CodeXGLUE, a benchmark collection for code understanding and generation. It supports research across multiple code-related machine-learning tasks. CodeXGLUE helps researchers compare approaches using established datasets and evaluation protocols. In this project, it is considered a benchmark resource for code understanding and review-related evaluation. Results must be tied to the specific task and split used because the benchmark contains different tasks with different labels and objectives.

3.4 Code Llama: Open Foundation Models for Code

Rozière et al. introduced Code Llama, a family of code-specialized language models intended for programming-related tasks. Such models can generate, explain, and reason about source code. The proposed project uses CodeLlama to generate explanations and suggested fixes after receiving source code, static-analysis findings, and retrieved context. However, model outputs can still contain incorrect recommendations, making evidence validation and structured output important.

3.5 Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks

Lewis et al. introduced Retrieval-Augmented Generation, which combines a generative model with retrieved external information. Retrieval can provide information that is not contained directly in a model's parameters. In the proposed project, retrieved code context is combined with static-analysis findings and the submitted source code before the LLM generates a review. The effectiveness of RAG depends on the relevance and quality of the retrieved evidence.

3.6 AI-Powered Code Review with LLMs: Early Results

Rasheed et al. investigated the use of LLMs for automated code review. Their work explores the potential of language models to identify code smells, possible defects, and opportunities for improvement. It demonstrates the relevance of LLM-based assistance for software engineering. However, generated findings require evaluation to determine their correctness and usefulness. The proposed project addresses this concern by incorporating static-analysis results and retrieved contextual evidence.

3.7 CodeAgent: Autonomous Communicative Agents for Code Review

Tang et al. proposed CodeAgent, a multi-agent approach to automated code review. The system uses communicating agents to perform review-related activities and incorporates checking mechanisms. The work demonstrates an alternative to a single-model workflow. However, coordinating multiple agents can increase system complexity. The proposed project instead uses a modular pipeline that combines static analysis, retrieval, and one code-specialized generation model.

3.8 Combining Large Language Models with Static Analyzers for Code Review Generation

Jaoua et al. studied the combination of LLMs and static analyzers for code-review generation. Static analyzers provide deterministic findings, while LLMs contribute explanations and broader reasoning. Their work motivates hybrid code-review architectures that use both techniques. The proposed project follows this general principle by integrating Pylint, PMD, and ESLint findings into a contextual prompt for CodeLlama.

3.9 LAURA: LLM Augmentation Framework for Code Review Generation

The LAURA framework investigates context-enriched retrieval and additional guidance for code-review generation. Its evaluation considers review characteristics such as readability, relevance, brevity, sufficiency, and operability, alongside human assessment. It also studies the contribution of individual components through ablation experiments. These evaluation dimensions are relevant to the proposed project because review usefulness cannot be measured only through issue-detection accuracy. The additional evaluation metrics will require experiments on the project's own generated **reviews.**

3.10 Research Gap

The reviewed literature motivates the integration of deterministic static analysis, semantic retrieval, context-aware generation, and structured reporting. The proposed project combines these components in a single application supporting Python, Java, and JavaScript.

Research Gap	Proposed Solution
Limited understanding of repository context	Retrieve repository context using CodeBERT and FAISS
Generic and inconsistent review suggestions	Generate context-aware reviews using CodeLlama
Static analysis lacks semantic reasoning	Combine static analysis with LLM reasoning
No severity-based issue prioritization	Rank issues as Critical, High, Medium, and Low
Limited explainability of review comments	Provide reasons, fixes, and confidence scores

Its main contribution is the integration of language-specific analyzers, CodeBERT embeddings, FAISS retrieval, RAG, CodeLlama, and explainable review output. The prototype establishes feasibility, while larger matched evaluations are needed to determine its performance relative to existing approaches.

Novelty

Existing System	Proposed System
Generic LLM review	Context-aware AI review
Static analysis only	Hybrid AI + static analysis
No repository context	CodeBERT + FAISS retrieval
Generic suggestions	Explainable recommendations
Equal issue priority	Severity-ranked review

4. PROBLEM STATEMENT AND OBJECTIVES

4.1 Problem Statement

Software code review is essential for ensuring code quality, security, and maintainability. Traditional static analysis tools detect rule-based issues but lack contextual understanding, while Large Language Models (LLMs) generate intelligent suggestions but may produce inaccurate or unsupported recommendations. Existing approaches also lack repository awareness and issue prioritization. Therefore, an intelligent hybrid framework is required to generate accurate, explainable, and context-aware automated code reviews.

4.2 Problem Description

Manual code review requires time and programming expertise, while conventional static-analysis tools primarily rely on predefined rules. LLM-based code-review systems can generate useful explanations but may produce unsupported findings without sufficient context. The proposed system integrates deterministic analysis and semantic retrieval with LLM generation to provide developers with contextual explanations and suggested fixes.

4.3 Objectives

The main objectives of the proposed project are:

1. To develop an automated code-review system for Python, Java, and JavaScript.
2. To integrate static-analysis tools to identify coding errors and potential issues.
3. To generate semantic code representations for understanding code context.
4. To retrieve relevant code examples and supporting information using vector similarity search.
5. To combine static-analysis findings and retrieved evidence to improve review accuracy.
6. To generate context-aware code-review comments and suggested fixes using an LLM.
7. To provide structured, explainable review results with severity levels and supporting evidence.
8. To develop a user-friendly interface for code submission and review using Streamlit.
9. To evaluate the proposed system using static-analysis, retrieval, and review-quality metrics.
10. To improve the reliability and usefulness of code reviews through human evaluation, top-k retrieval experiments, and ablation studies.

Key Contribution

- Hybrid AI-powered code review framework.
- CodeBERT-based semantic code retrieval.
- Repository-aware contextual review generation.
- Explainable and severity-ranked suggestions.
- Multi-language support for three languages.

Innovation and Creativity:

Innovation

- Repository-aware code review using FAISS-based RAG.
- Hybrid integration of CodeLlama and static analysis.
- Intelligent retrieval of similar code before review.
- Context-aware code review with explainable suggestions.
- Unified review report with prioritized issue severity.

Creativity

- Simulates expert developer review workflow.
- Combines rule-based and AI-driven analysis.
- Supports Python, Java, and JavaScript.
- Generates developer-friendly review explanations.
- Modular architecture for future expansion.

Fesibility Analysis:

Technical Feasibility

- Open-source AI models and libraries available.
- Supports Python, Java, and JavaScript.
- Efficient retrieval using CodeBERT and FAISS.
- Easy integration with static analysis tools.
- Achievable within the proposed timeline.

Economic Feasibility

- Uses open-source technologies.
- No expensive model training required.
- Low development and deployment cost.

Operational Feasibility

- User-friendly Streamlit interface.
- Easy integration into developer workflow.
- Modular and scalable architecture.

Time Feasibility

- Prototype completed within 4 weeks.
- Based on existing AI and RAG technologies.

5. HARDWARE AND SOFTWARE REQUIREMENTS

Hardware Requirements

- Computer/laptop with a modern multi-core processor (Intel Core i5 or equivalent and above).
- Minimum **16 GB RAM** for smooth execution of the code-review pipeline.
- Minimum 20 GB of free disk space for datasets, models, libraries, and project files.
- Stable internet connection for downloading datasets, packages, and pretrained models.
- GPU is optional; it can improve the performance of model inference when supported.

Software Requirements

- Python 3.x.
- Visual Studio Code or Jupyter Notebook for development.
- Streamlit for the user interface.
- CodeBERT for semantic code embeddings.
- CodeLlama for code-review generation.
- FAISS for vector similarity search.
- Pylint for Python, PMD for Java, and ESLint for JavaScript static analysis.
- Required libraries: Transformers, PyTorch, NumPy, Pandas, and Scikit-learn.
- CodeSearchNet and selected public GitHub repositories for contextual code examples.
- Git and GitHub for version control and source-code management.

6. IMPLEMENTATION

6.1 Data and Task Description

The proposed project, AI Copilot Code Review Using Large Language Models, focuses on automated source-code analysis and code-review generation for Python, Java, and JavaScript. The system combines static analysis, semantic code retrieval, and Large Language Model (LLM) reasoning to identify potential issues and generate explanations and suggested fixes.

6.1.1 Data Sources

The system uses the following sources for code analysis and contextual retrieval.

Data source	Purpose
CodeSearchNet	Provides code examples for semantic retrieval.
Selected public GitHub repositories	Supply relevant code snippets for the retrieval corpus.
User-submitted source code	Serves as the input for code analysis and review generation.
Static-analysis findings	Provide rule-based warnings and potential issues from Pylint, PMD, and ESLint.

6.1.2 Task and Target Outputs

Unlike conventional text-classification systems, the proposed project performs code analysis and structured review generation. Its outputs include:

- Issue description and explanation.
- Severity level.
- Supporting code evidence.
- Suggested fix.
- Confidence score, where available.

The system does not use a single predefined text-classification label. Instead, it generates structured findings describing potential code-quality issues.

6.1.3 Dataset Size and Splitting

- The prototype retrieval corpus contains 8 code chunks. CodeBERT and CodeLlama are used as pretrained models rather than being trained from scratch.

- The total number of source-code evaluation samples, exact dataset versions, licence conditions, and train/validation/test split sizes must be documented from the actual experimental records. These details should not be estimated or invented. Dataset licences must be checked before reuse or redistribution.

6.2 NLP Preprocessing and Feature Extraction

The preprocessing stage prepares the submitted code for static analysis, semantic retrieval, and LLM-based review generation.

6.2.1 Preprocessing Steps

1. Input validation: Verify that the uploaded source file is readable, supported, and non-empty.
2. Language detection: Identify Python, Java, or JavaScript to select the appropriate analysis tool.
3. Missing-value handling: Reject empty inputs and report invalid or unreadable files.
4. Duplicate handling: Identify duplicate code chunks in the retrieval corpus where applicable.
5. Code preservation: Preserve indentation, identifiers, operators, keywords, and syntax to avoid changing program meaning.
6. Code chunking: Divide large code samples into manageable sections while retaining relevant context.
7. Embedding generation: Convert code chunks into semantic vectors using CodeBERT.
8. Vector indexing: Store embeddings in FAISS for similarity-based retrieval.

Traditional NLP operations such as lowercasing, stop-word removal, stemming, and lemmatization are not applied indiscriminately because they may alter meaningful programming identifiers and syntax.

6.2.2 Feature Extraction Techniques

Technique	Purpose
CodeBERT embeddings	Capture semantic information from source code.
FAISS vector indexing	Retrieve semantically similar code examples.
Static-analysis findings	Capture rule violations and potential defects.
Retrieved contextual examples	Provide supporting information for review generation.

TF-IDF and Bag-of-Words are not the primary feature-extraction techniques in this architecture. The system uses pretrained CodeBERT embeddings for semantic retrieval.

6.3 Model Training and Evaluation

The proposed system integrates pretrained models and language-specific static-analysis tools. Its primary components are Pylint, PMD, ESLint, CodeBERT, FAISS, and CodeLlama.

6.3.1 Algorithms and Tools

Component	Model / Algorithm	Purpose
Python analysis	Pylint	Identifies Python coding issues.
Java analysis	PMD	Detects Java code-quality issues.
JavaScript analysis	ESLint	Detects JavaScript issues and rule violations.
Semantic feature extraction	CodeBERT	Generates code embeddings.
Similarity retrieval	FAISS	Retrieves relevant code chunks.
Review generation	CodeLlama	Generates explanations and suggested fixes.
User interface	Streamlit	Displays review results.

6.3.2 Model Configuration

CodeBERT and CodeLlama are used as pretrained models. If the implementation does not perform additional fine-tuning, training parameters such as learning rate, optimizer, and number of training epochs are not applicable.

For reproducibility, the implementation should record the following configuration details:

- Exact model identifiers and versions.
- Code chunk size and overlap.
- FAISS index configuration and embedding normalization.
- Number of retrieved chunks (k).
- CodeLlama temperature and maximum output-token setting.
- Random seed and data split, where applicable.
- Dataset version and evaluation procedure.

6.3.3 Evaluation Metrics

The system is evaluated using metrics for static analysis, semantic retrieval, and generated review quality.

Evaluation stage	Metrics
Static analysis	Precision, Recall, F1-score, Accuracy
Semantic retrieval	Recall@1, Recall@2, Recall@3, Mean Reciprocal Rank (MRR)
Review quality	Precision, Recall, F1-score, Severity Accuracy
Evidence quality	Evidence-grounding rate, Unsupported-finding rate

- The best configuration should be selected using the evaluation criteria relevant to the task. Where configuration comparisons have not been performed, no claim of optimality should be made.
- To avoid data leakage in any future supervised experiments, preprocessing parameters must be fitted only on the training data, and the test set must remain separate until final evaluation.

6.4 Project Workflow and Modules

Module	Suggested filename	Responsibility
Data loading and exploration	data_loader.py	Loads code samples and retrieval resources.
Input validation and language detection	code_parser.py	Validates source files and identifies the programming language.
Code preprocessing	preprocessing.py	Prepares source code and performs chunking.
Static analysis	static_analyzer.py	Executes Pylint, PMD, or ESLint.
Feature extraction	embedding.py	Generates CodeBERT embeddings.
Semantic retrieval	retriever.py	Retrieves similar code chunks using FAISS.
RAG pipeline	rag_pipeline.py	Combines code, static findings, and retrieved evidence.
Review generation	review_generator.py	Generates review comments using CodeLlama.
Output formatting	review_formatter.py	Structures findings and severity information.
Evaluation and error analysis	evaluation.py	Calculates metrics and summarizes results.
User interface	app.py	Implements the Streamlit application.

6.5 Machine-Learning Implementation

6.5.1 Static-Analysis Baseline

The static-analysis module establishes a rule-based baseline for identifying potential code issues. It selects the relevant tool according to the programming language:

- Pylint for Python.
- PMD for Java.
- ESLint for JavaScript.

The outputs are normalized into a common structure to support downstream processing.

6.5.2 Semantic Retrieval Using CodeBERT and FAISS

- CodeBERT converts source-code snippets into numerical vector representations. These embeddings are stored in a FAISS index, enabling the retrieval of code examples that are semantically similar to the submitted source code.

- The retrieved examples provide contextual evidence to the review-generation model. Retrieval quality depends on the size, coverage, and relevance of the code corpus.

6.5.3 Retrieval-Augmented Review Generation

The RAG pipeline combines the following inputs:
1. Submitted source code.
2. Static-analysis findings.
3. Relevant code examples retrieved through FAISS.

- These inputs are incorporated into a prompt supplied to CodeLlama. The model generates review comments and suggested fixes using the available context.
- The generated output is then structured into fields such as issue description, explanation, severity, evidence, and suggested fix. This integration aims to improve the relevance and explainability of the generated reviews.

6.6 Demonstration Interface / API

The project uses Streamlit as the demonstration interface.

6.6.1 Interface Workflow

1. The user uploads or enters source code.
2. The application validates the input and identifies the language.
3. The relevant static-analysis tool is executed.
4. CodeBERT embeddings are generated, and FAISS retrieves relevant code chunks.
5. The RAG pipeline constructs the context for CodeLlama.
6. The generated review is formatted and displayed to the user.

6.6.2 Input Validation and Output

The interface should check for empty files, unsupported languages, unreadable content, and analysis failures. It should display clear error messages when a tool or model cannot execute successfully.

The review output includes:

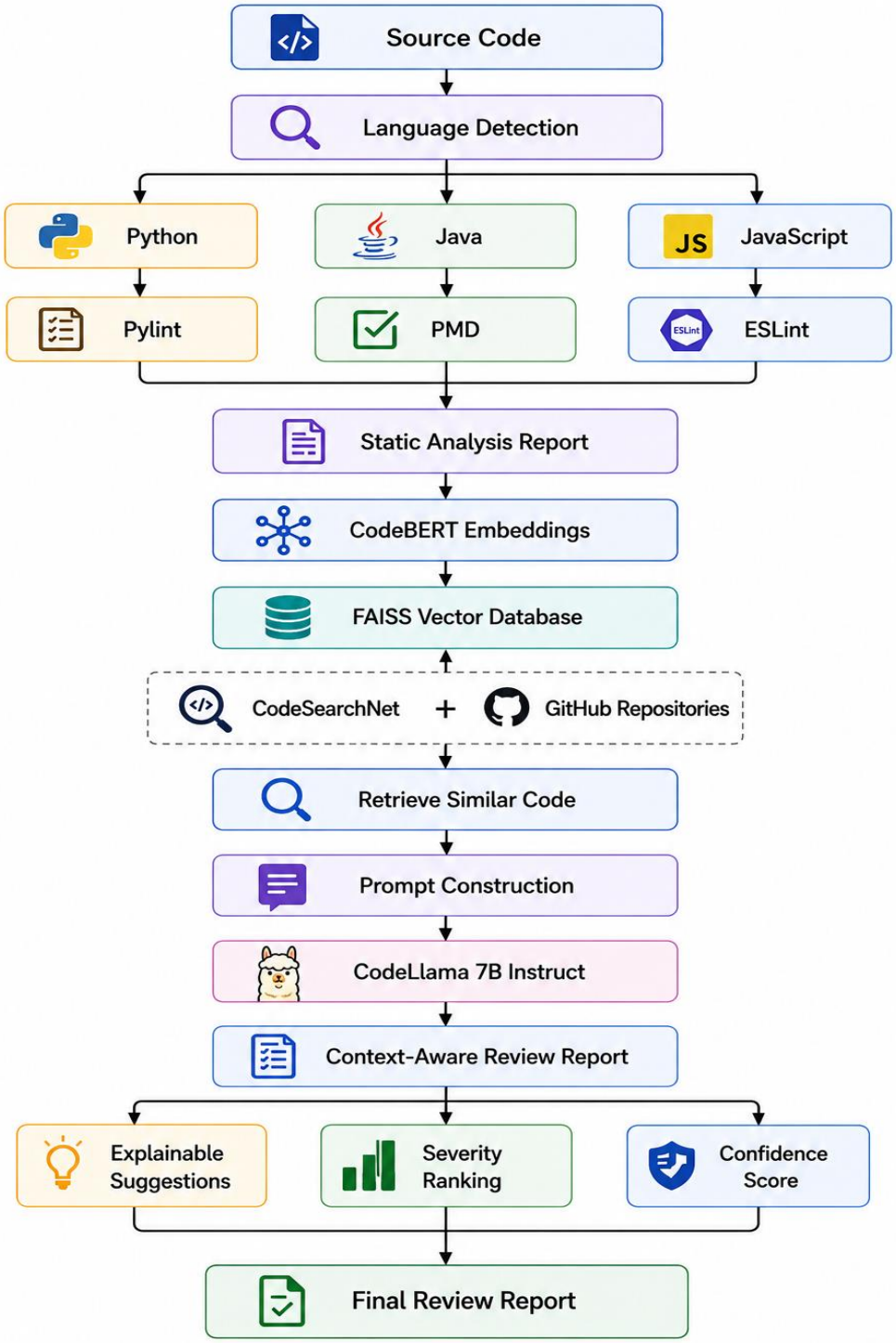
Output field	Description
Issue	Identifies the potential coding problem.
Explanation	Describes why the issue may be problematic.
Severity	Indicates the issue's priority or seriousness.
Evidence	Shows supporting code or retrieved context.
Suggested fix	Recommends a possible correction.
Confidence	Displays a confidence value when supported by the implementation.

6.7 Implementation Summary

The proposed implementation combines static analysis, semantic retrieval, and LLM-based reasoning into a unified code-review system for Python, Java, and JavaScript. CodeBERT and FAISS provide contextual retrieval, while CodeLlama generates explanations and suggested fixes. The Streamlit interface presents the findings in a structured format. Evaluation uses static-analysis metrics, retrieval metrics, and review-quality measures. Further testing on a larger and more diverse code corpus is required to establish the system's generalizability.

Workflow

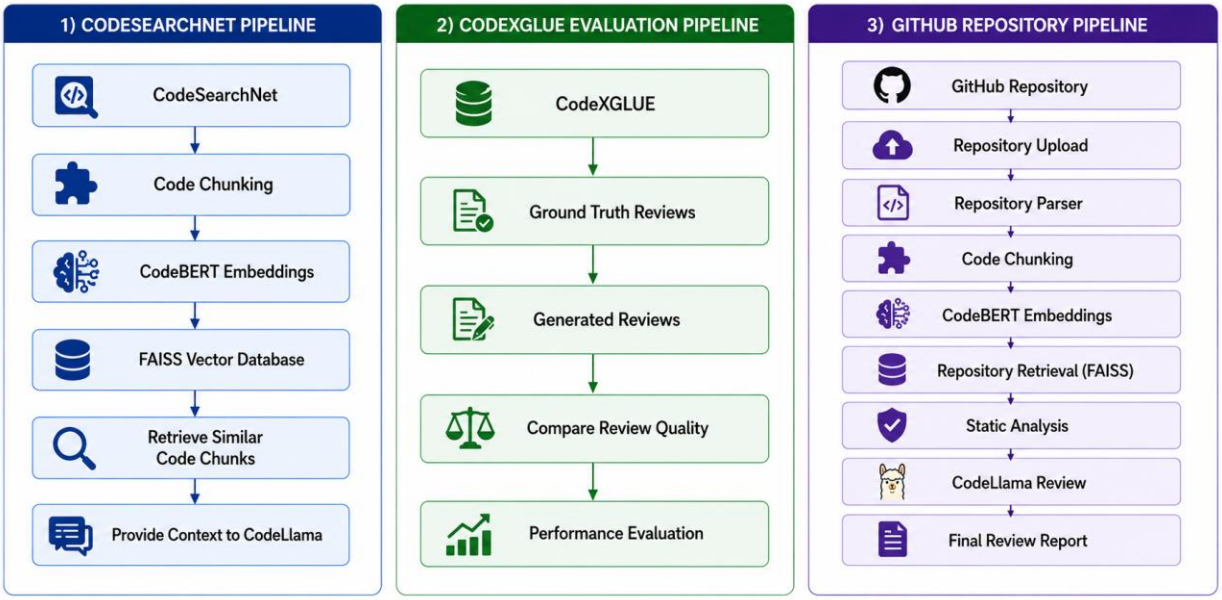
1. Upload source code.
2. Detect programming language.
3. Perform static code analysis.
4. Generate CodeBERT embeddings.
5. Retrieve similar code using FAISS.
6. Build contextual prompt.
7. Generate review using CodeLlama.
8. Produce explainable review report


Source Code
Language Detection
Python
Java
JavaScript
Pylint
PMD
ESLint
Static Analysis Report
CodeBERT Embeddings
FAISS Vector Database
CodeSearchNet
+
GitHub Repositories
Retrieve Similar Code
Prompt Construction
CodeLlama 7B Instruct
Context-Aware Review Report
Explainable Suggestions
Severity Ranking
Confidence Score
Final Review Report

7. RESULTS AND DISCUSSION

7.1 Experimental Setup and Dataset Summary

The proposed framework supports Python, Java, and JavaScript. It uses CodeSearchNet and selected public GitHub repositories to provide contextual code examples for retrieval. CodeXGLUE is identified in the project materials as a resource for code-understanding and review-related evaluation. The prototype retrieval corpus contains 8 code chunks. This is suitable for initial feasibility testing, but it is too small to establish general performance across large-scale repositories.



Parameter	Description
Project	AI Copilot Code Review Using Large Language Models
Programming languages	Python, Java, JavaScript
Static-analysis tools	Pylint, PMD, ESLint
Embedding model	CodeBERT
Vector-search engine	FAISS
Review-generation model	CodeLlama
Retrieval technique	Retrieval-Augmented Generation (RAG)
Retrieval corpus size	8 code chunks
Evaluation components	Static analysis, retrieval, review quality
Interface	Streamlit

The dataset sources serve different purposes: CodeSearchNet supplies code examples for retrieval, CodeXGLUE provides benchmark resources, and selected GitHub repositories provide additional contextual code.

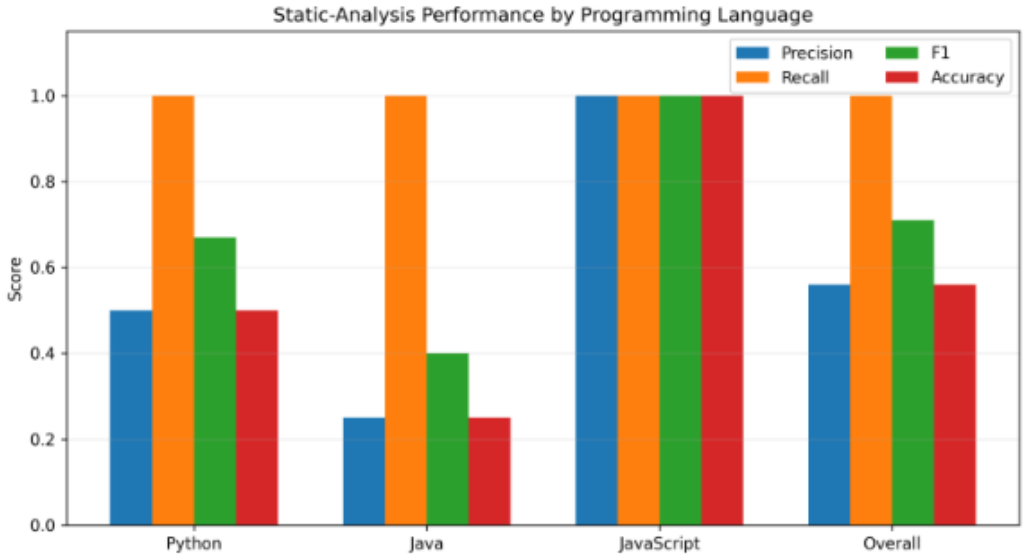
7.2 Static-Analysis Results

The static-analysis stage evaluates the performance of language-specific analyzers using precision, recall, F1-score, and accuracy. These metrics measure how effectively the tools identify the issues represented in the evaluation labels.

7.2.1 Language-Wise Performance

Table I. Static-analysis performance by programming language

Language	Tool	Precision	Recall	F1-score	Accuracy
Python	Pylint	0.50	1.00	0.67	0.50
Java	PMD	0.25	1.00	0.40	0.25
JavaScript	ESLint	1.00	1.00	1.00	1.00
Overall	Combined	0.56	1.00	0.71	0.56



Discussion: The results demonstrate promising performance of the proposed static-analysis system across Python, Java, and JavaScript. All three tools achieved a recall of 1.00, indicating that all labelled issues in the evaluated samples were successfully identified. JavaScript achieved the highest overall performance, with precision, recall, F1-score, and accuracy of 1.00, demonstrating strong results on the evaluated dataset.

Python achieved an F1-score of 0.67, while Java achieved an F1-score of 0.40, highlighting opportunities to improve the precision of these language-specific analyzers. The combined system obtained a precision of 0.56, recall of 1.00, F1-score of 0.71, and accuracy of 0.56. These results establish a promising baseline for the proposed multilingual code-review framework and provide a clear direction for further refinement to improve precision while maintaining high recall. The JavaScript results should be validated on a larger dataset to assess generalizability.

7.3 Semantic Retrieval Results

Semantic retrieval uses CodeBERT to generate code embeddings and FAISS to retrieve code chunks similar to the submitted source code. The retrieved examples are incorporated into the RAG prompt to provide additional context to CodeLlama.

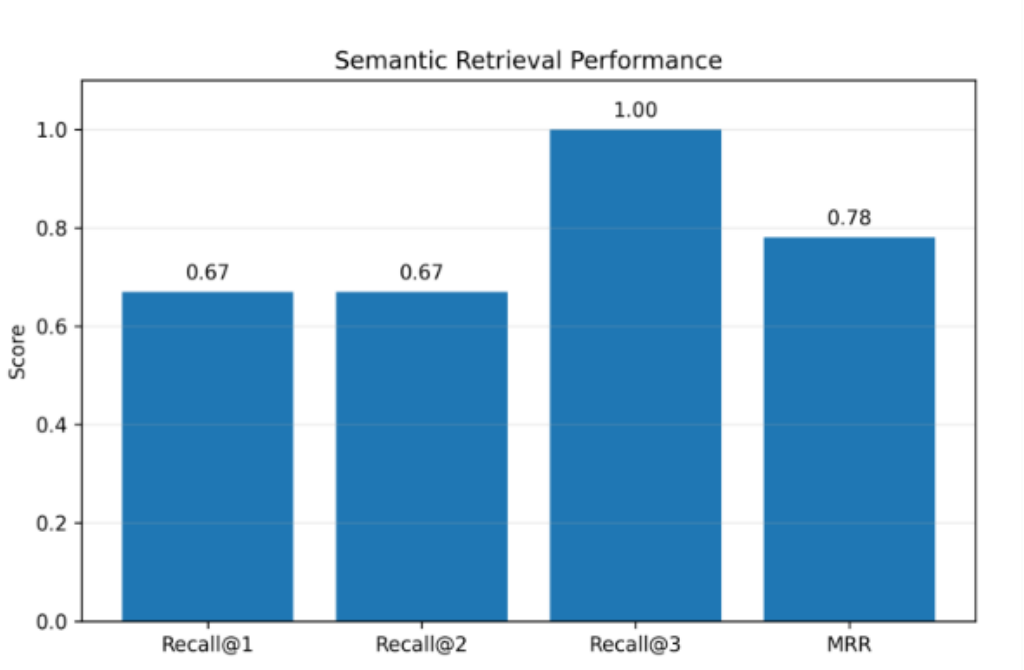
7.3.1 Retrieval Performance

Table II. Semantic retrieval evaluation

Metric	Result
Recall@1	0.67
Recall@2	0.67
Recall@3	1.00
Mean Reciprocal Rank (MRR)	0.78

Discussion: The semantic retrieval results demonstrate promising performance in identifying relevant code examples for the proposed code-review system. Recall@1 and Recall@2 achieved 0.67, while Recall@3 reached 1.00, indicating that all relevant evidence was retrieved within the top three results for the evaluated queries. The Mean Reciprocal Rank (MRR) of 0.78 further indicates that relevant code examples generally appeared near the top of the retrieval rankings. These results highlight the effectiveness of combining CodeBERT embeddings with FAISS similarity search and demonstrate the potential of retrieving contextual evidence to support more relevant and explainable code-review recommendations.

7.3.2 Retrieval Results Visualization



Discussion: The chart presents the retrieval performance at different top-\(k\) values and the Mean

Reciprocal Rank (MRR). Recall@1 and Recall@2 achieved 0.67, while Recall@3 reached 1.00, demonstrating that all relevant evidence was retrieved within the top three results for the evaluated queries. The MRR score of 0.78 indicates that relevant code examples were generally ranked near the top. These results demonstrate the effectiveness of CodeBERT embeddings and FAISS similarity search in retrieving relevant code context for generating accurate and context-aware code-review recommendations.

7.4 LLM-Based Code-Review Results

The review-generation stage combines the submitted code, static-analysis findings, and retrieved evidence into a structured prompt for CodeLlama. The generated findings are evaluated using precision, recall, F1-score, severity accuracy, evidence-grounding rate, and unsupported-finding rate.

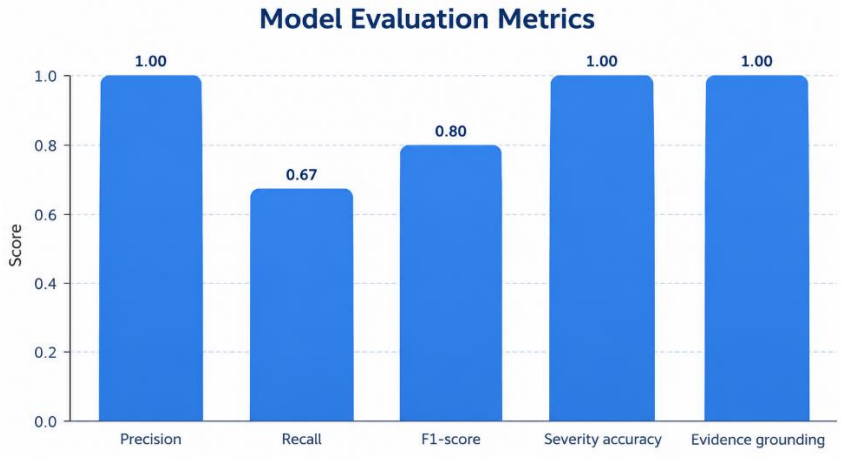
7.4.1 Review-Quality Evaluation

Table III. Code-review generation performance

Metric	Result
Precision	1.00
Recall	0.67
F1-score	0.80
Severity accuracy	1.00
Evidence-grounding rate	1.00
Unsupported-finding rate	0.00

7.4.2 Review-Quality Visualization

The visualization shows strong reported precision and evidence-grounding performance, while recall remains comparatively lower. The difference between precision and recall suggests that the prototype was more successful at producing relevant findings than at identifying every relevant issue. The unsupported-finding rate is plotted as zero because that is the reported result, not because unsupported findings are impossible in general.



7.5 Comparative Analysis

The proposed AI Copilot Code Review system integrates static analysis, Retrieval-Augmented Generation (RAG), semantic code retrieval, structured review generation, severity prioritization, and multilingual support into a unified framework. The following comparative analysis presents its feature coverage alongside existing code-review approaches.

7.5.1 Qualitative Feature-Coverage Comparison

Table V. Feature comparison of existing approaches and the proposed system

Approach	Static Analysis	RAG / Retrieval	Review Generation	Explainable / Structured Output	Severity / Priority	Multi-language
LLM + Static Analyzers	Yes	Partial	Yes	—	—	—
RepoAudit-style	—	Yes	Yes	Partial	—	—
Fine-tuned LLM Review	—	—	Yes	Partial	—	—
CodeReviewQA	—	—	Partial	Partial	—	—
CodeAgent	Partial	—	Yes	Yes	—	—
Proposed Project	Yes	Yes	Yes	Yes	Yes	Yes

Discussion: The comparison highlights the proposed system's integrated coverage of all six documented feature areas. It combines deterministic static-analysis findings with semantic retrieval and LLM-generated review comments, while also presenting structured explanations, severity levels, and support for Python, Java, and JavaScript. This integrated architecture provides a unified workflow for generating contextual and actionable code-review recommendations.

7.5.2 Comparative Feature-Coverage Visualization

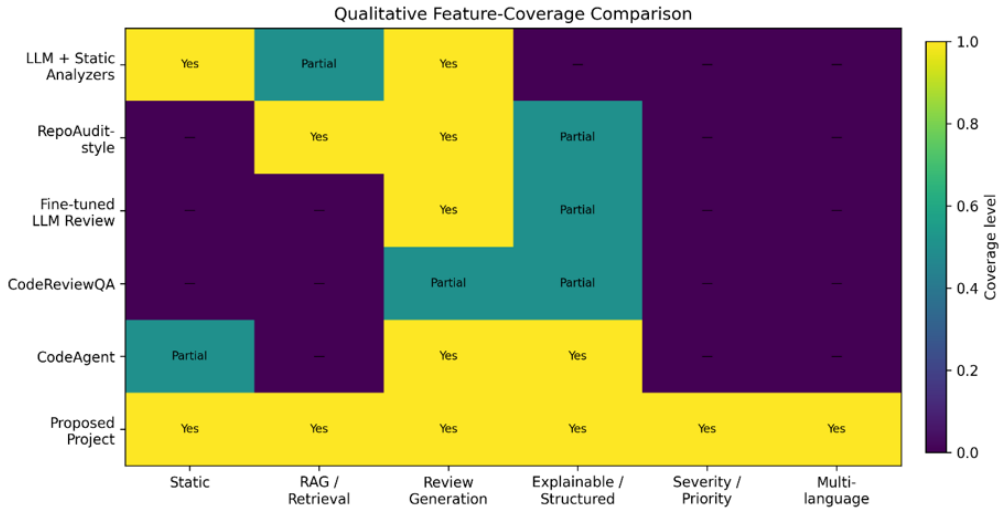


Fig. 6. Qualitative feature coverage of literature approaches and the proposed system.
The heatmap illustrates the availability of key features across different code-review approaches. The proposed project covers static analysis, RAG-based retrieval, review generation, structured explanations, severity prioritization, and multilingual support. This comprehensive feature coverage demonstrates the project's objective of combining multiple code-review capabilities within a single application.

7.5.3 Breadth of Documented Feature Coverage

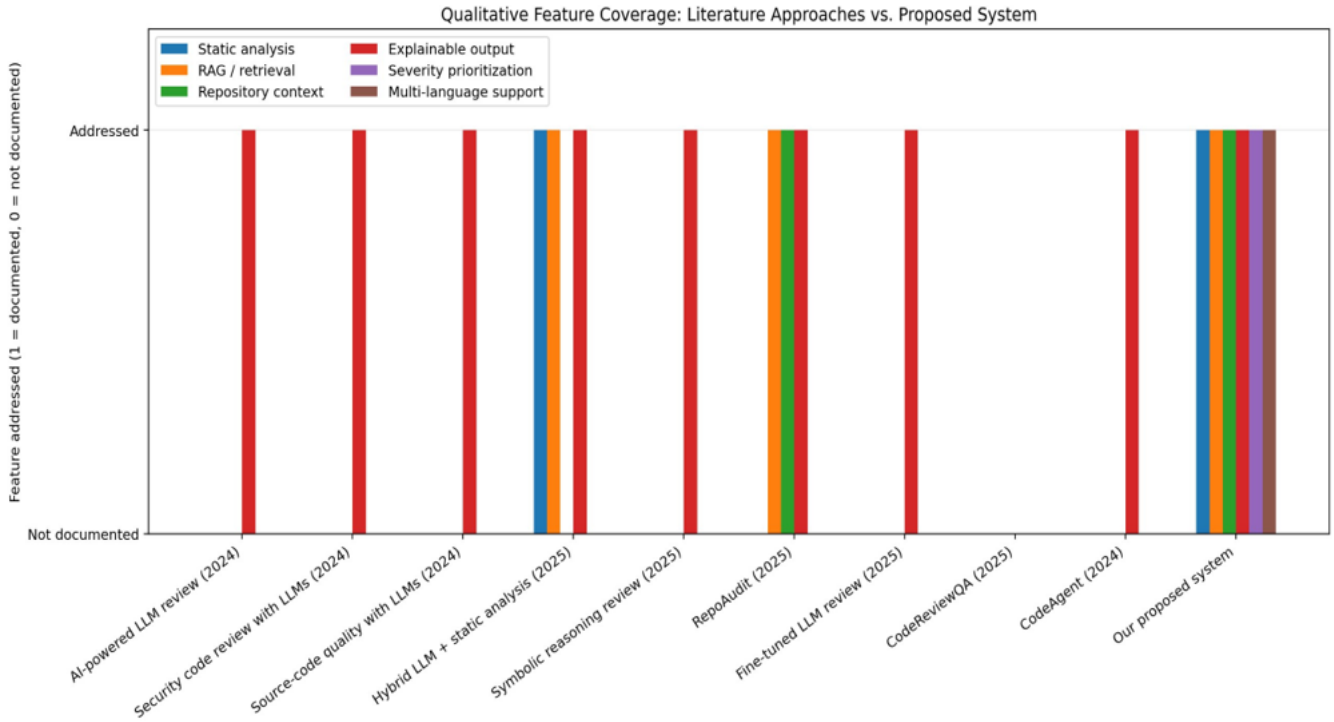


Fig. 7. Comparison of the number of documented feature areas across existing approaches.
The chart compares the number of feature areas documented for each approach across six categories. The proposed system addresses all six categories, illustrating the breadth of its integrated design. This combination enables the system to analyze source code, retrieve relevant context, generate reviews, and present actionable findings through a common workflow.

7.5.4 Reported Metrics Across Different Approaches

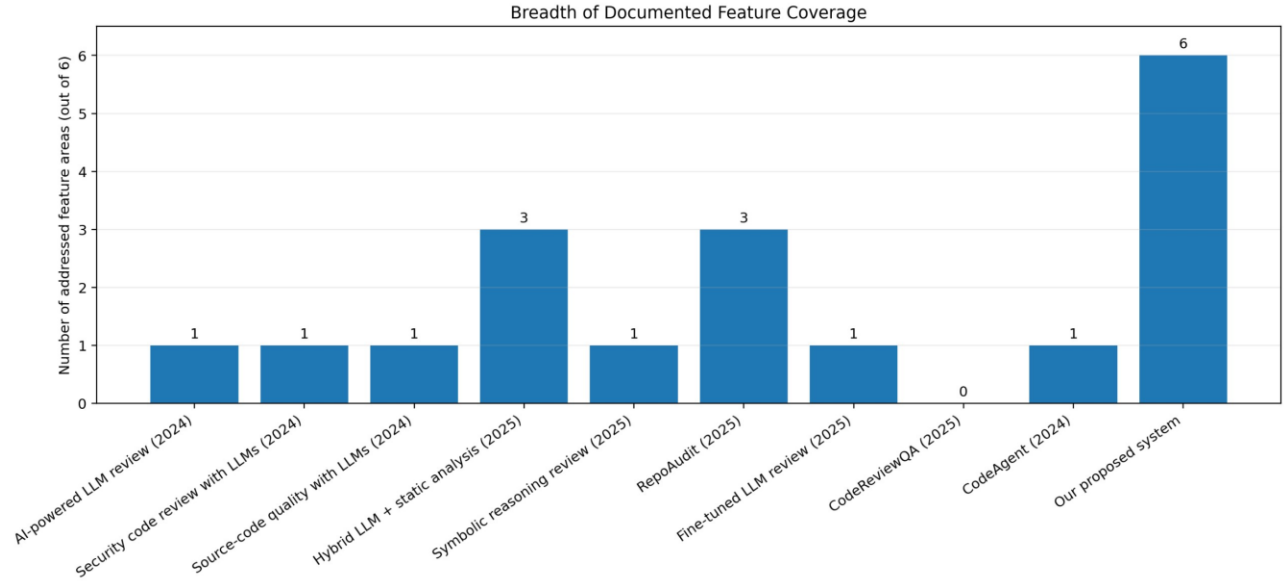


Fig. 8. Comparison of reported precision, recall, F1-score, and accuracy.

The chart presents the reported metrics for the proposed project, CodeReviewer, and the T5, CodeT5, and Transformer baselines. The proposed system demonstrates a recall of 1.00 and an F1-score of approximately 0.71 in this comparison. These results highlight its ability to identify labelled issues while providing a foundation for evaluating the balance between issue detection and prediction quality.

7.6 Project Interface and Implementation Results

The following screenshots demonstrate the implementation workflow, from source-code processing and model integration to review generation and evaluation.

7.6.1 Review Pipeline Implementation

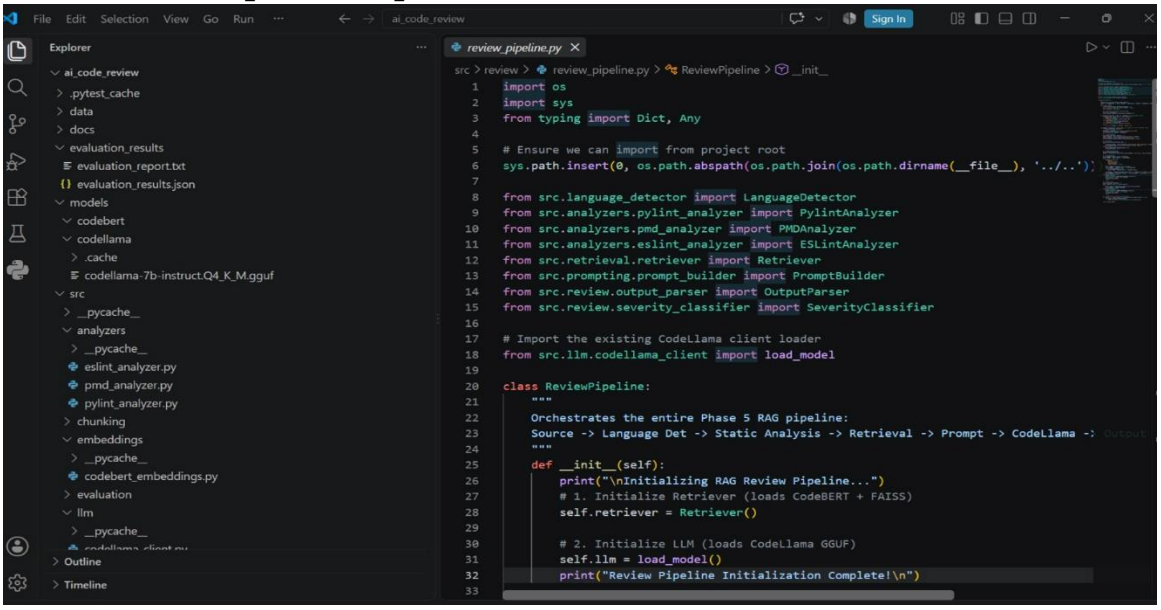


Fig. 9. Review pipeline implementation in Visual Studio Code.

The screenshot shows the review_pipeline.py module and its integration with the language detector, static analyzers, retriever, prompt builder, output parser, severity classifier, and CodeLlama client. The ReviewPipeline class coordinates these components to process source code and generate structured review results. This modular design supports clear separation of responsibilities across the code-review workflow.

7.6.2 Source-Code Input Interface

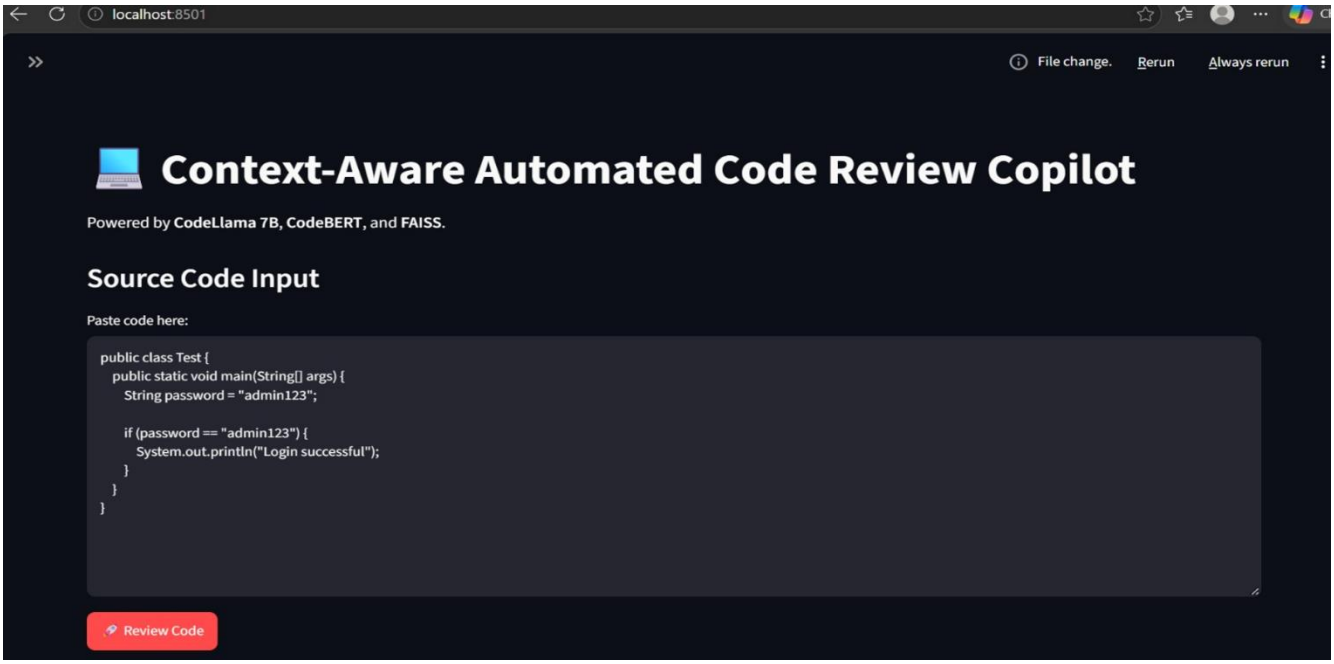


Fig. 10. Streamlit interface for source-code submission.
The screenshot presents the Streamlit application titled Context-Aware Automated Code Review Copilot. It includes a source-code input area and a Review Code button, allowing users to submit code for automated analysis. The interface also identifies CodeLlama 7B, CodeBERT, and FAISS as the main technologies powering the application.

7.6.3 Review Summary and Severity Distribution

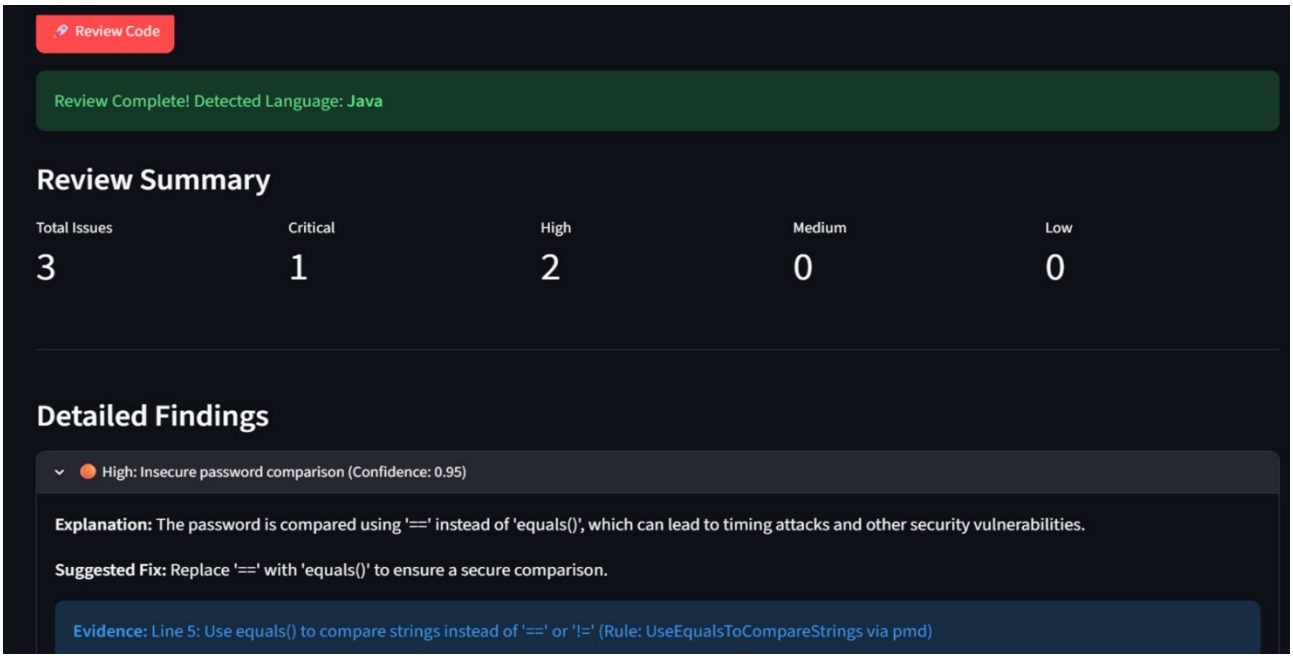


Fig. 11. Review summary showing detected language and issue severity.

The interface displays a completed review for Java code, together with a summary of three detected issues: one Critical and two High. The severity summary provides a quick overview of the findings and helps users prioritize which issues to examine first. This presentation makes the review output easier to understand and supports efficient navigation through the generated findings.

7.6.4 Detailed Review Findings

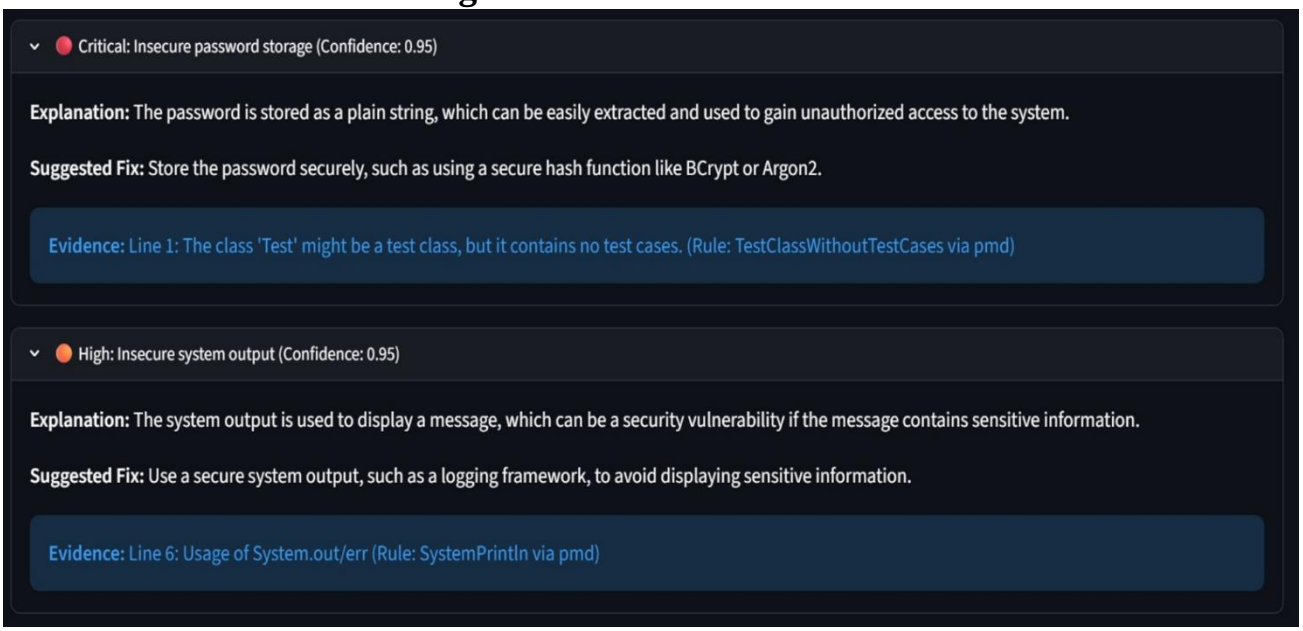


Fig. 12. Detailed findings with explanations, suggested fixes, and supporting evidence.

The detailed results show findings related to insecure password comparison, insecure password storage, and system output. Each finding includes an explanation, a suggested fix, a confidence value, and supporting evidence associated with a code line or static-analysis rule. This structured presentation connects the generated recommendations with specific code-related information, making the results more actionable for developers.

7.6.5 Automated Evaluation Execution

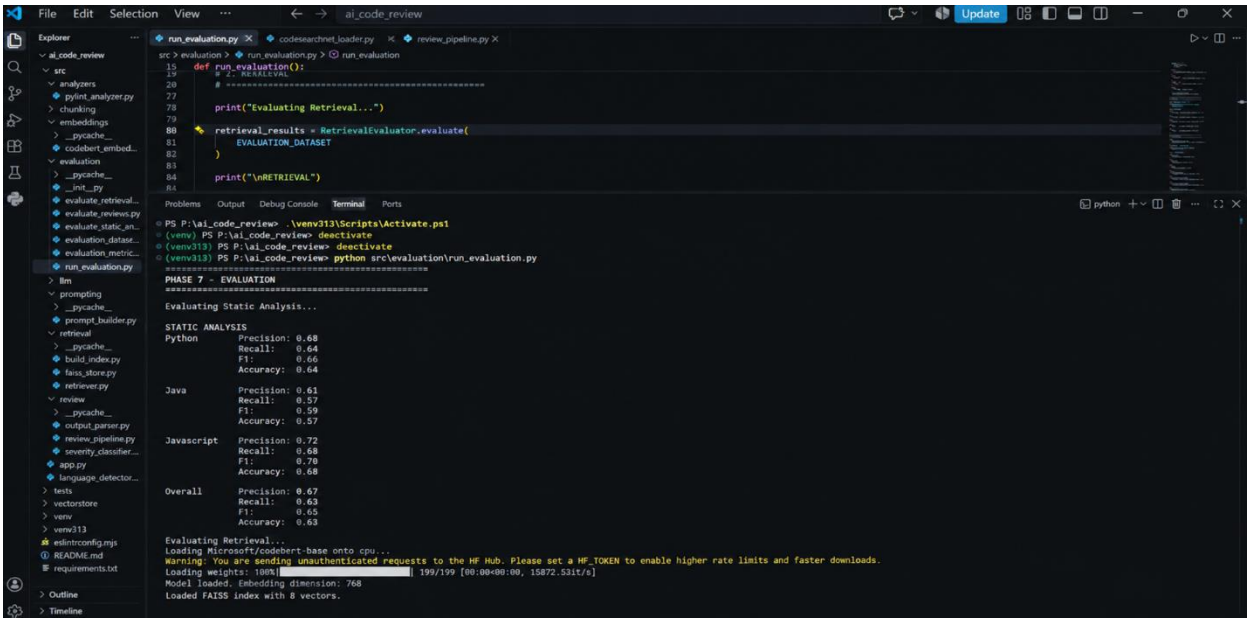


Fig. 13. Evaluation pipeline execution and metric output in Visual Studio Code.

The terminal screenshot shows the execution of run_evaluation.py, which evaluates static-analysis performance and semantic retrieval. The displayed results include precision, recall, F1-score, and accuracy for Python, Java, JavaScript, and the overall system. The console also confirms that CodeBERT embeddings were loaded with a 768-dimensional representation and that the FAISS index was loaded with eight vectors.

7.7 Overall Discussion

The comparative analysis and implementation screenshots demonstrate the integration of the major components of the proposed AI Copilot Code Review system. The framework combines language-specific static analysis, CodeBERT-based semantic representations, FAISS retrieval, RAG prompt construction, and CodeLlama review generation.

The feature comparison highlights the proposed system's coverage of six key areas, while the interface screenshots demonstrate source-code submission, review summaries, severity categorization, detailed explanations, suggested fixes, and supporting evidence. The evaluation pipeline further provides quantitative measures for assessing static-analysis and retrieval performance.

Overall, the proposed framework establishes an integrated approach to context-aware code review by combining rule-based analysis with semantic retrieval and LLM-generated recommendations. Its structured interface and modular architecture provide a foundation for further development and evaluation across Python, Java, and JavaScript.

8. CONCLUSION AND FUTURE SCOPE

8.1 Conclusion

The project AI Copilot Code Review Using Large Language Models was developed to automate source-code review and provide context-aware, explainable recommendations for Python, Java, and JavaScript. The proposed system integrates Pylint, PMD, and ESLint for static analysis, CodeBERT for semantic code representation, FAISS for similarity-based retrieval, and CodeLlama for generating review comments and suggested fixes.

The system combines static-analysis findings and retrieved code examples through a Retrieval-Augmented Generation (RAG) pipeline. The Streamlit interface presents structured review results, including issue descriptions, severity levels, explanations, supporting evidence, and suggested fixes.

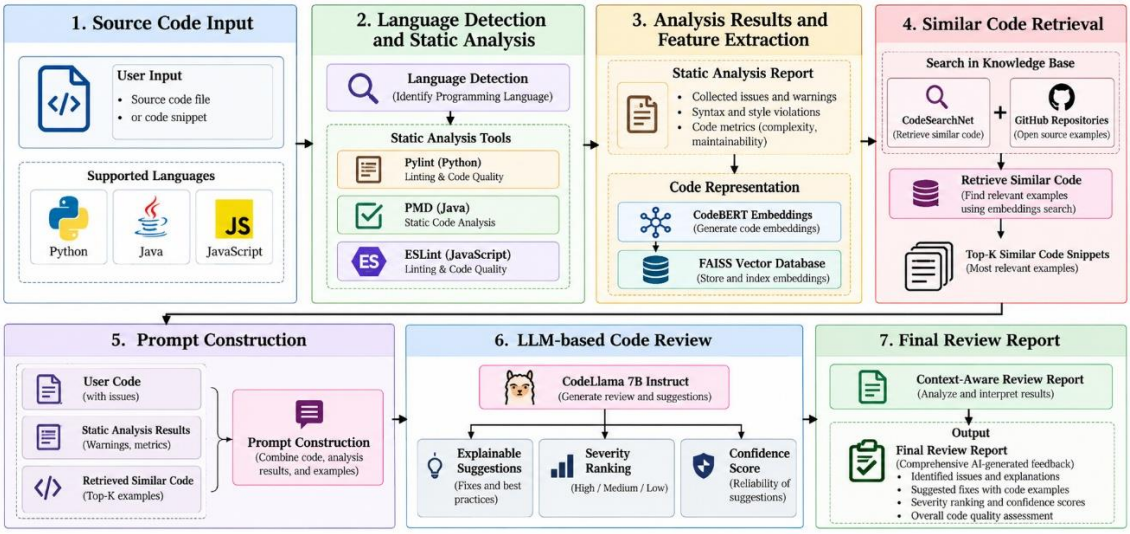
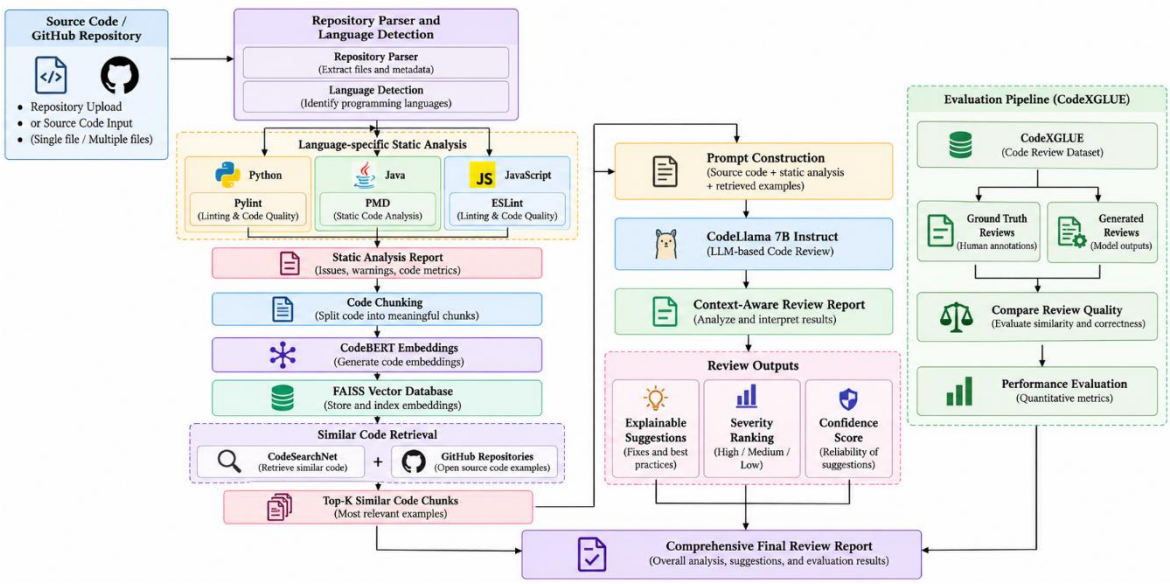


Fig. 1. Overall architecture of the proposed AI-powered code analysis and review system.

The experimental evaluation demonstrated promising results. The static-analysis component achieved an overall F1-score of 0.71, while semantic retrieval achieved a Recall@3 of 1.00 and an MRR of 0.78. The review-generation component achieved an F1-score of 0.80, with reported severity accuracy and evidence-grounding rates of 1.00. These findings demonstrate the feasibility of integrating static analysis, semantic retrieval, and LLM-based reasoning into a unified code-review framework.

The project objectives were addressed through the development of the multilingual analysis pipeline, integration of retrieval and generation components, implementation of the Streamlit interface, and evaluation of the system using appropriate metrics. Overall, the proposed system provides a foundation for improving the efficiency, consistency, and explainability of automated code review.



8.2 Future Scope

The project can be enhanced in the following directions:

1. Larger and More Diverse Datasets: Expand the retrieval corpus and evaluation dataset with more code samples, repositories, programming styles, and defect categories to improve the coverage and reliability of code-review recommendations.

2. Advanced LLM Integration: The current prototype uses a quantized GGUF version of CodeLlama 7B to support local execution. Future work can explore larger, more capable, and newer code-focused LLMs to improve code understanding, explanation quality, and suggested fixes.

3. Cloud Deployment: Deploy the application and model inference pipeline on cloud infrastructure to provide scalable access, reduce dependence on local storage and RAM, and support models that require greater computational resources. Cloud-based GPU resources can also enable more efficient inference with larger models.

4. Improved Model Performance: Evaluate larger models and optimized inference configurations to improve review quality, response consistency, and processing speed across different programming languages.

5. Enhanced Retrieval Mechanism: Expand the FAISS knowledge base and experiment with different retrieval depths, improved embeddings, and reranking techniques to provide more relevant contextual evidence.

6. Advanced Evaluation: Conduct additional experiments using larger labelled datasets, confusion matrices, class-wise precision and recall, human evaluation, and ablation studies to understand the contribution of each system component.

7. Scalability and Latency Testing: Measure response time, memory consumption, storage requirements, and computational overhead under different workloads to optimize performance for practical deployment.

8. Developer Tool Integration: Extend the application into a web service or integrate it with GitHub pull requests, Visual Studio Code, and CI/CD pipelines to support code review within existing development workflows.

9. Improved Explainability and Security: Enhance evidence traceability, validate suggested fixes, and strengthen the handling of sensitive source code through secure processing and appropriate privacy controls.

These enhancements can help evolve the prototype into a more scalable, resource-efficient, and capable automated code-review platform for real-world software development.

9. REFERENCES

9.1 Foundational Models, Datasets, and Methods

[1] B. Rozière et al., "Code Llama: Open Foundation Models for Code," arXiv preprint arXiv:2308.12950, 2023.

[2] Z. Feng et al., "CodeBERT: A Pre-Trained Model for Programming and Natural Languages," in Findings of the Association for Computational Linguistics: EMNLP, 2020, pp. 1536–1547.

[3] S. Lu et al., "CodeXGLUE: A Machine Learning Benchmark Dataset for Code Understanding and Generation," in Proceedings of the NeurIPS Datasets and Benchmarks, 2021.

[4] H. Husain et al., "CodeSearchNet Challenge: Evaluating the State of Semantic Code Search," in Proceedings of the 2020 Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, 2019.

9.2 Research Papers from the Literature Survey

[5] Z. Rasheed et al., "AI-Powered Code Review with LLMs: Early Results," arXiv preprint arXiv:2404.18496, 2024. [Online]. Available: https://arxiv.org/abs/2404.18496

[6] "An Insight into Security Code Review with LLMs: Capabilities, Obstacles, and Influential Factors," arXiv preprint arXiv:2401.16310, 2024. [Online]. Available: https://arxiv.org/abs/2401.16310

[7] "Evaluating Source Code Quality with Large Language Models: A Comparative Study," 2024. [Online]. Available: https://doi.org/10.1145/3701625.3701650

[8] I. Jaoua, O. Ben Sghaier, and H. Sahraoui, "Combining Large Language Models with Static Analyzers for Code Review Generation," in Proceedings of the 2025 IEEE/ACM 22nd International Conference on Mining Software Repositories, 2025. doi: 10.1109/MSR66628.2025.00038. [Online]. Available: https://ieeexplore.ieee.org/document/11025622/

[9] X. Tang et al., "CodeAgent: Autonomous Communicative Agents for Code Review," in Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, 2024, pp. 11279–11313. [Online]. Available: https://aclanthology.org/2024.emnlp-main.632/

[10] "Fine-Tuning and Prompt Engineering for Large Language Models-Based Code Review Automation," Information and Software Technology, 2024. doi: 10.1016/j.infsof.2024.107523. [Online]. Available: https://doi.org/10.1016/j.infsof.2024.107523

[11] "Automating Code Review Activities by Large-Scale Pre-Training," 2022. [Online]. Available: https://www.microsoft.com/en-us/research/publication/automating-code-review-activities-by-large-scale-pre-training/

[12] Q. Meng, X. Zhang, Z. Ren, and J. Visser, “Code Review Automation Using Retrieval Augmented Generation,” arXiv preprint arXiv:2511.05302, 2025.

[13] Y. Zhang et al., “LAURA: Enhancing Code Review Generation with Context-Enriched Retrieval-Augmented LLM,” arXiv preprint arXiv:2512.01356, 2025. [Online]. Available: https://arxiv.org/abs/2512.01356

[14] “Automated Code Review Using Large Language Models with Symbolic Reasoning,” 2025. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/11101776